

PayFlow API Assignment Write-up

Student Details

Name: Siddhant Darekar

Repository Link: https://github.com/Siddhant20104111/payflow-api/tree/development/src/main/java/com/payflow/payflow_api

Task 2: Database Creation

User Table Creation SQL

create table transaction (id bigint generated by default as identity, amount float(53), receiver_upi_id varchar(255), sender_upi_id varchar(255), timestamp timestamp(6), primary key (id))

Transaction Table Creation SQL

Hibernate: create table users (id bigint generated by default as identity, balance float(53), name varchar(255), phone_number varchar(255), upi_id varchar(255), primary key (id))

H2 Console Screenshots

Users Table

The screenshot displays the H2 Console interface. On the left, a tree view shows the database structure: jdbc:h2:mem:payflowdb, TRANSACTION, and USERS. The USERS table is expanded, showing columns: ID, BALANCE, NAME, PHONE_NUMBER, UPI_ID, and Indexes. The main panel shows the SQL statement 'SELECT * FROM USERS' being executed. The results pane below shows the query was successful, returning 'no rows' in '2 ms'. The results are displayed in a table with columns: ID, BALANCE, NAME, PHONE_NUMBER, and UPI_ID. An 'Edit' button is visible below the results.

Run Run Selected Auto complete Clear SQL statement:

SELECT * FROM USERS

SELECT * FROM USERS;

ID	BALANCE	NAME	PHONE_NUMBER	UPI_ID
(no rows, 2 ms)				

Edit

Transactions Table

The screenshot shows a database client interface with a sidebar on the left displaying the database structure. The main area on the right shows a SQL query and its results.

Database Structure (Left Sidebar):

- jdbc:h2:mem:payflowdb
 - TRANSACTION
 - ID
 - AMOUNT
 - RECEIVER_UPI_ID
 - SENDER_UPI_ID
 - TIMESTAMP
 - Indexes
 - USERS
 - INFORMATION_SCHEMA
 - Users

SQL Statement:

```
SELECT * FROM TRANSACTION TRANSACTION
```

Results:

ID	AMOUNT	RECEIVER_UPI_ID	SENDER_UPI_ID	TIMESTAMP
(no rows, 5 ms)				

Buttons: Run, Run Selected, Auto complete, Clear, Edit

Task 5: Endpoint Testing

Create User 1

Command:

```
curl -X POST http://localhost:8080/users -H "Content-Type: application/json" -d '{"name":"Rahul","upiId":"rahul@upi","phoneNumber":"9999999999","balance":5000}'
```

Output:

```
{"id":1,"name":"Rahul","upiId":"rahul@upi","phoneNumber":"9999999999","balance":5000.0}
```

Create User 2

Command:

```
curl -X POST http://localhost:8080/users -H "Content-Type: application/json" -d '{"name":"Priya","upiId":"priya@upi","phoneNumber":"8888888888","balance":3000}'
```

Output:

```
{"id":2,"name":"Priya","upild":"priya@upi","phoneNumber":"8888888888","balance":3000.0}
```

Get All Users

Command:

```
curl http://localhost:8080/users
```

Output:

```
[{"id":1,"name":"Rahul","upild":"rahul@upi","phoneNumber":"9999999999","balance":5000.0}, {"id":2,"name":"Priya","upild":"priya@upi","phoneNumber":"8888888888","balance":3000.0}]
```

Get User By ID

Command:

```
curl http://localhost:8080/users/1
```

Output:

```
{"id":1,"name":"Rahul","upild":"rahul@upi","phoneNumber":"9999999999","balance":5000.0}
```

Find User By UPI

Command:

```
curl http://localhost:8080/users/upi/rahul@upi
```

Output:

```
{"id":1,"name":"Rahul","upild":"rahul@upi","phoneNumber":"9999999999","balance":5000.0}
```

Send Money

Command:

```
curl -X POST http://localhost:8080/transactions -H "Content-Type: application/json" -d '{"senderUpild":"rahul@upi","receiverUpild":"priya@upi","amount":1000}'
```

Output:

{ "id":1,"senderUpild":"rahul@upi","receiverUpild":"priya@upi","amount":1000.0,"timestamp":"2026-06-13T21:29:34.8400468" }

Database After Transactions

Users Table

jdbc:h2:mem:payflowdb

TRANSACTION

USERS

INFORMATION_SCHEMA

Users

H2 2.4.240 (2025-09-22)

RunRun SelectedAuto completeClearSQL statement:

SELECT * FROM USERS

SELECT * FROM USERS;

ID	BALANCE	NAME	PHONE_NUMBER	UPI_ID
1	5000.0	Rahul	9999999999	rahul@upi
2	3000.0	Priya	8888888888	priya@upi

(2 rows, 3 ms)

Edit

Transactions Table

jdbc:h2:mem:payflowdb

TRANSACTION

USERS

INFORMATION_SCHEMA

Users

H2 2.4.240 (2025-09-22)

RunRun SelectedAuto completeClearSQL statement:

SELECT * FROM TRANSACTION

SELECT * FROM TRANSACTION;

ID	AMOUNT	RECEIVER_UPI_ID	SENDER_UPI_ID	TIMESTAMP
1	1000.0	priya@upi	rahul@upi	2026-06-13 21:29:34.840047

(1 row, 1 ms)

Edit

Demonstration of @RequestBody

Without @RequestBody:

```
curl -X POST http://localhost:8080/users/test -H "Content-Type: application/json" -d
{"name":"Rahul"}
```

Output:

```
{"id":null,"name":null,"upild":null,"phoneNumber":null,"balance":null}
```

With @RequestBody:

```
curl -X POST http://localhost:8080/users/test2 -H "Content-Type: application/json" -d
{"name":"Rahul"}
```

Output:

```
{"id": null,"name": "Rahul","upild": null,"phoneNumber": null,"balance": null}
```

Explanation:

Without @RequestBody, Spring MVC does not read the JSON payload from the HTTP request body. It attempts to bind request parameters instead, resulting in all fields being null. When @RequestBody is used, Jackson reads the JSON payload and converts it into a User object by matching JSON keys with Java field names.

Conceptual Answers

1. Request Lifecycle

When curl sends a POST request to /users, the request first reaches the embedded Tomcat server. Tomcat forwards the request to Spring MVC's DispatcherServlet, which acts as the front controller. The DispatcherServlet identifies the correct controller method through handler mappings and uses a HandlerAdapter to invoke it. The User object is created from the JSON request body, and the controller method executes before returning a response.

2. Serialization

Spring Boot uses the Jackson library to convert incoming JSON into Java objects. When the JSON payload contains keys such as name and upild, Jackson maps them to matching Java fields. If the JSON key is written as upi_id instead of upild, Jackson will not automatically

map it unless additional configuration or annotations are provided. As a result, the field may remain null.

3. Spring Boot Features

The first feature is the embedded server, where Tomcat is started automatically without separate installation. The second feature is auto-configuration, which configures Hibernate, JPA, H2 Database, and Spring MVC automatically based on dependencies. The third feature is production-ready defaults, which provide logging, error handling, dependency management, and sensible application configuration.

4. Spring vs Spring Boot

Using plain Spring would require manually configuring the servlet container, DispatcherServlet, database connections, JPA settings, and dependency versions. Spring Boot eliminates most of this setup through auto-configuration and starter dependencies. It also provides an embedded server and simplified project setup. As a result, development becomes faster and less error-prone.

5. Stateless REST

Stateless means that each HTTP request contains all information needed to process it, and the server does not remember previous requests. The POST /transactions endpoint processes each request independently. This becomes important when multiple servers are placed behind a load balancer because any server can handle any request without requiring shared session data.

6. Persistence

If transactions were stored in a Java List, all records would be lost when the application restarts because memory is cleared. Databases persist data on storage and allow records to survive application shutdowns. This is essential for payment systems because transaction history must never disappear. Losing transaction records would make auditing and financial tracking impossible.